CSC 421 - Deep Learning 1

Instructor: Shengyao Lu

Readings

CHAPTER 21 DEEP LEARNING

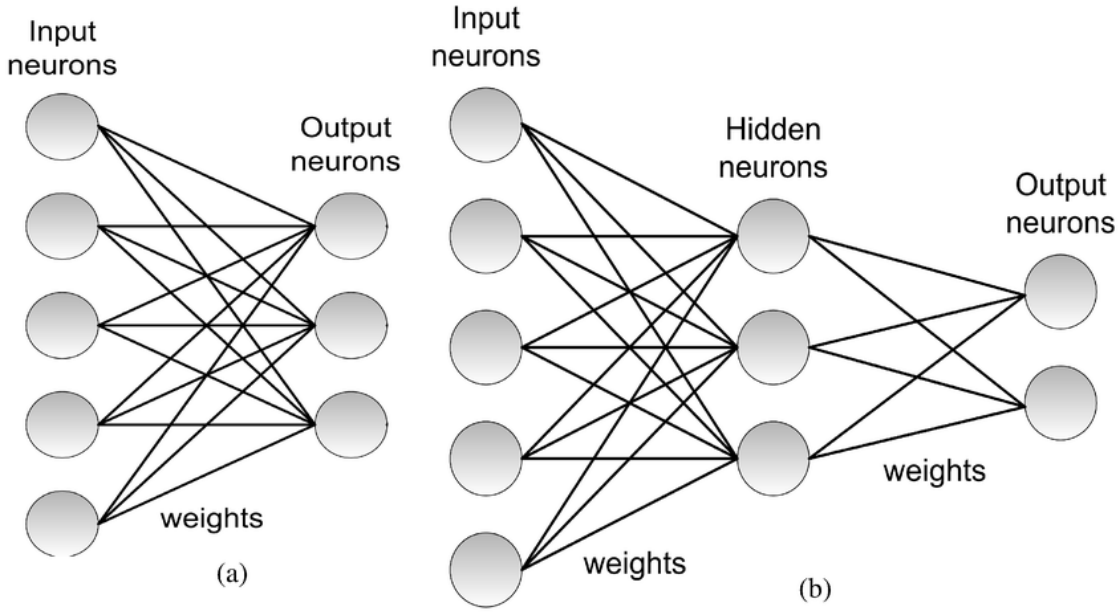
- Basic: Sections 21.1, 21.2, and Summary
- Expected: Same as Basic plus 21.3
- Advanced: All the chapter including bibliographical and historical notes

1. Artificial Neural Networks $ANNs/NNs$

- Neural networks represent complex nonlinear functions by composing multiple layers of parameterized units.
- A **multilayer neural network** or multilayer perceptron MLP is an artificial neural network with an input layer, an output layer, and at least one hidden layer.
  - While we provide inputs and outputs to train the model, we, the humans, don't provide any values to the units inside the hidden layers.
  - Each unit in the first hidden layer receives a weighted value from each of the units in the input layer, performs some action on it and outputs a value.
  - Each of these values is weighted and further propagated to the next layer, repeating the process until the output layer is reached.
  - Through hidden layers, it is possible to model non-linear data.

Characteristics of ANNs

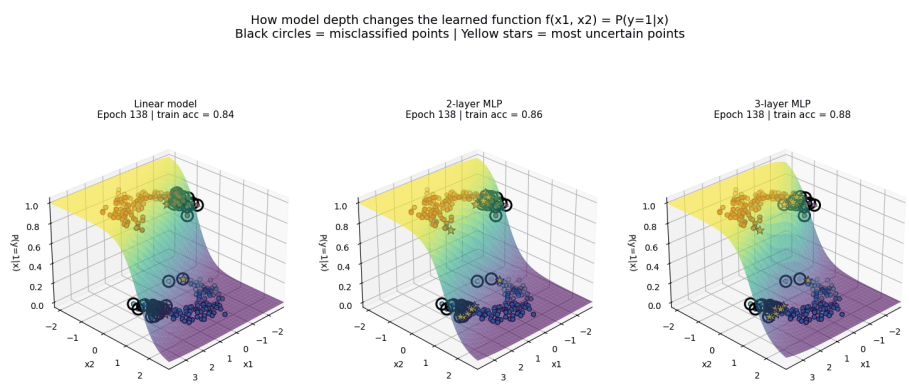
- Slow convergence - training
- Prone to overfitting due to the large number of parameters
- Can solve complex problems in addition to classification
- Unclear how to find the optimal architecture for the hidden layer$s$
- Easy to parallelize - deep learning resurgence has been enabled, among other things, by GPU processing



Drawbacks of Linear models

Although linear and logistic regression can handle a large number of input variables,

- the computation path from each input to the output is very short:
  - multiplication by a single weight,
  - then adding into the aggregate output.
- the different input variables contribute independently to the output, without interacting with each other



MLP

$$h^{(i+1)} = a^{(i)}(W^{(i)}h^{(i)} + b^{(i)})$$

- $h^{(i)}$: the layer representation at Layer i
- $W^{(i)}$: weight matrix at Layer i
- b: bias
- a: activation function
- input: $h^{(0)} = x$
- output: $\hat{y} = h^{(L)}$ for a L-layer MLP

Activation Function

- The **universal approximation theorem** states that:
  - a feed-forward neural network with at least one hidden layer, a finite number of neurons, and a non-linear activation function can approximate any continuous, compact function to any desired precision.
- Without activation functions, multiple linear layers is the equivalent to a single linear layer. $simplelineartransformation$
- Activation functions introduce nonlinearity, making universal approximation possible in NNs.
- Typical activation functions
  - sigmoid function

$$\sigma(x) = \frac{1}{1 + e^{-x}}$$

  - ReLU $akarectifiedlinearunit$

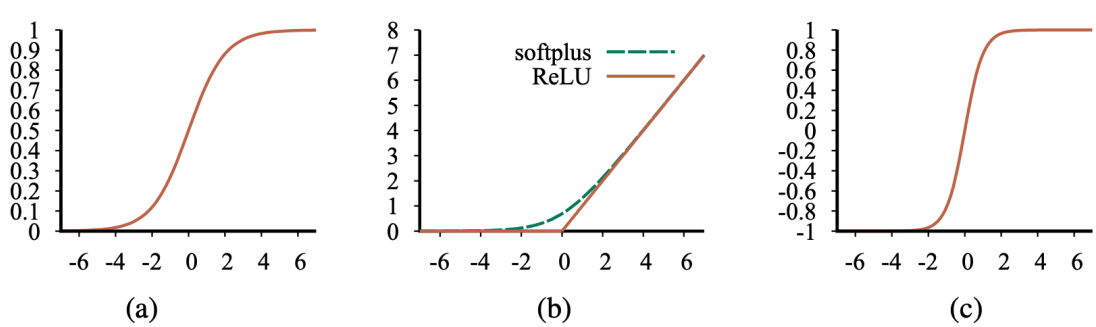
$$\mathrm{ReLU}(x) = \max(0, x)$$

  - softplus $smoothversionofReLU$

$$\mathrm{softplus}(x) = \log(1 + e^{x})$$

  - tanh

$$\tanh(x) = \frac{e^{2x} - 1}{e^{2x} + 1}$$



2. Backpropagation

- Backpropagation applies the chain rule from the output layer back to the input layer.
- Backpropagation does not change the learning rule. It only provides an efficient way to compute gradients.
  - gradient descent = learning rule
  - backprop = gradient computation

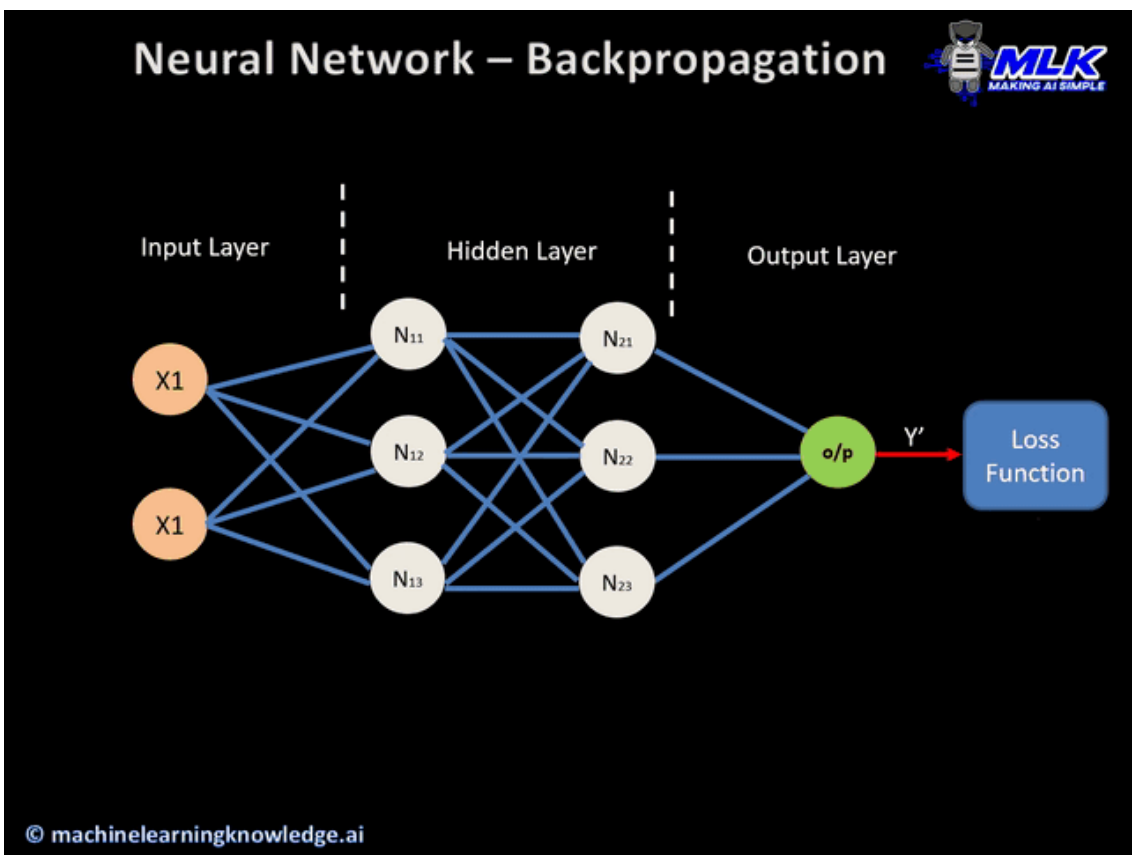
Example

$$\hat{y} = f_3(f_2(f_1(x)))$$

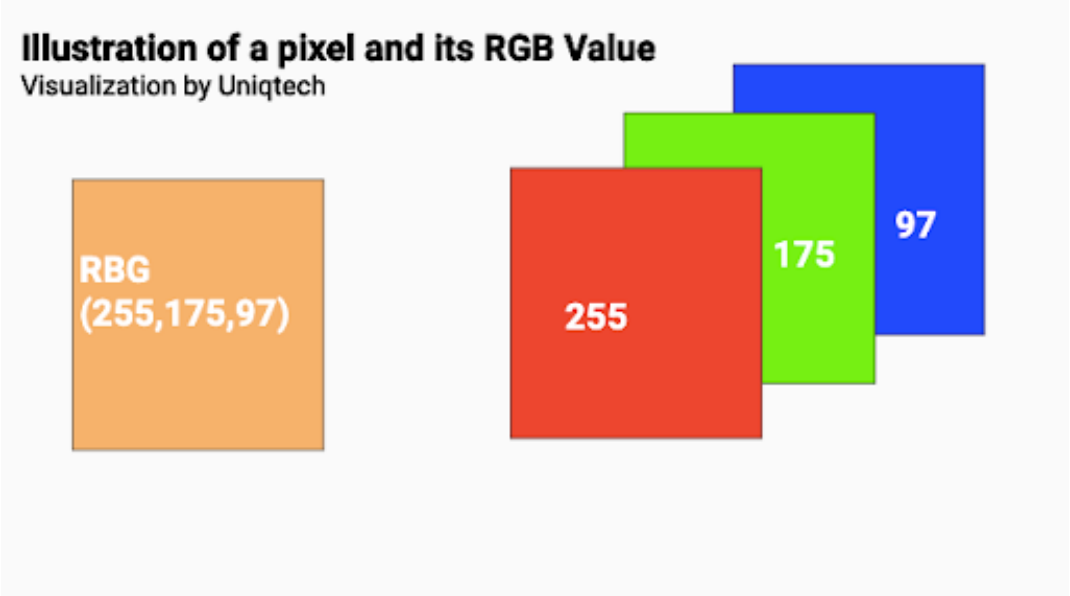
$$\hat{y} = \sigma(W_3 \, \mathrm{ReLU}(W_2 \, \mathrm{ReLU}(W_1 x)))$$

$$\frac{\partial \mathcal{L}}{\partial W_1} = \frac{\partial \mathcal{L}}{\partial \hat{y}} \frac{\partial \hat{y}}{\partial h_2} \frac{\partial h_2}{\partial h_1} \frac{\partial h_1}{\partial W_1}$$

- direct calculations of gradients can be complicated, backpropagation avoids repeated computation and computes all gradients in time proportional to the forward pass.



3. Computer vision



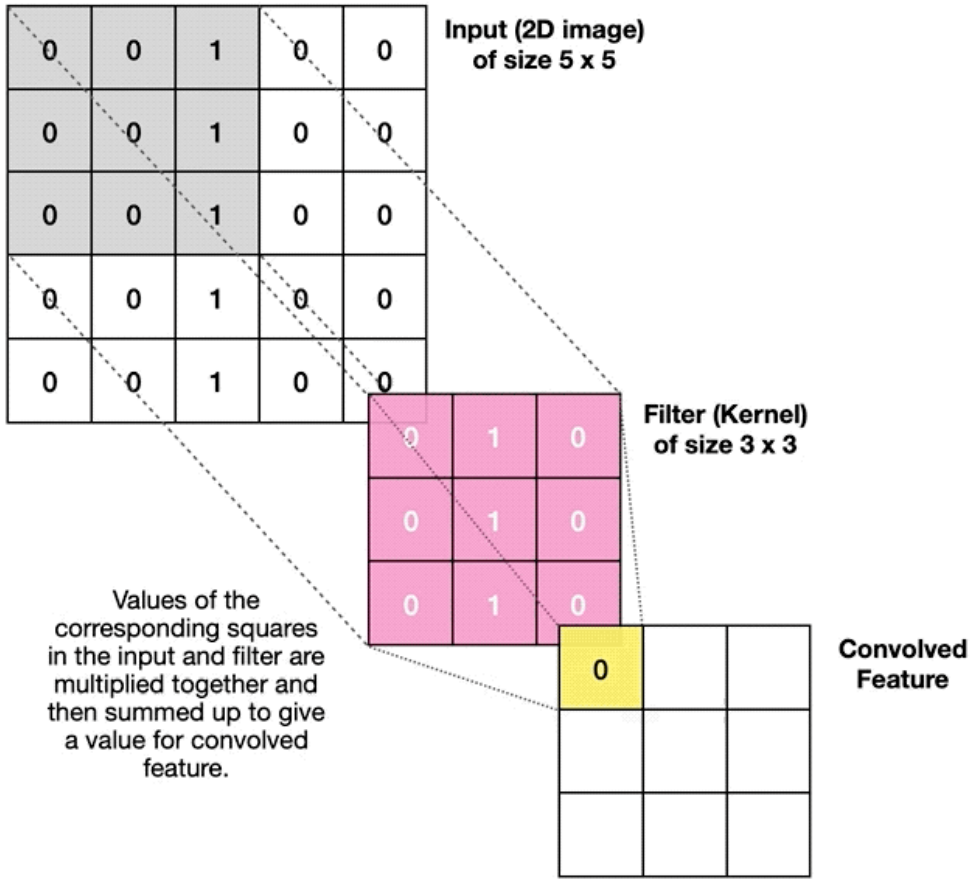
Computer vision encompasses the different computational methods for analyzing and understanding digital images, and it is often achieved using neural networks. For example, computer vision is used when social media employs face recognition to automatically tag people in pictures. Other examples are handwriting recognition and self-driving cars.

Images consist of pixels, and pixels are represented by three values that range from 0 to 255, one for red, one for green and one for blue. These values are often referred to with the acronym RGB. We can use this to create a neural network where each color value in each pixel is an input, where we have some hidden layers, and the output is some number of units that tell us what it is that was shown in the image.

However, there are a few drawbacks to this approach.

1. by breaking down the image into pixels and the values of their colors, **we can't use the structure of the image as an aid**. That is, as humans, if we see a part of a face we know to expect to see the rest of the face, and this quickens computation. We want to be able to use a similar advantage in our neural networks.
2. the sheer number of inputs is very big, which means that we will have to calculate a lot of weights.

Image Convolution



$source: OpenCourseWareCS50'sIntroductiontoArtificialIntelligencewit$

Image convolution is applying a filter that adds each pixel value of an image to its neighbors, weighted according to a kernel matrix. Doing so alters the image and can help the neural network process it.

10	20	30	40
10	20	30	40
20	30	40	50
20	30	40	50

0	-1	0
-1	5	-1
0	-1	0

10	20
40	50

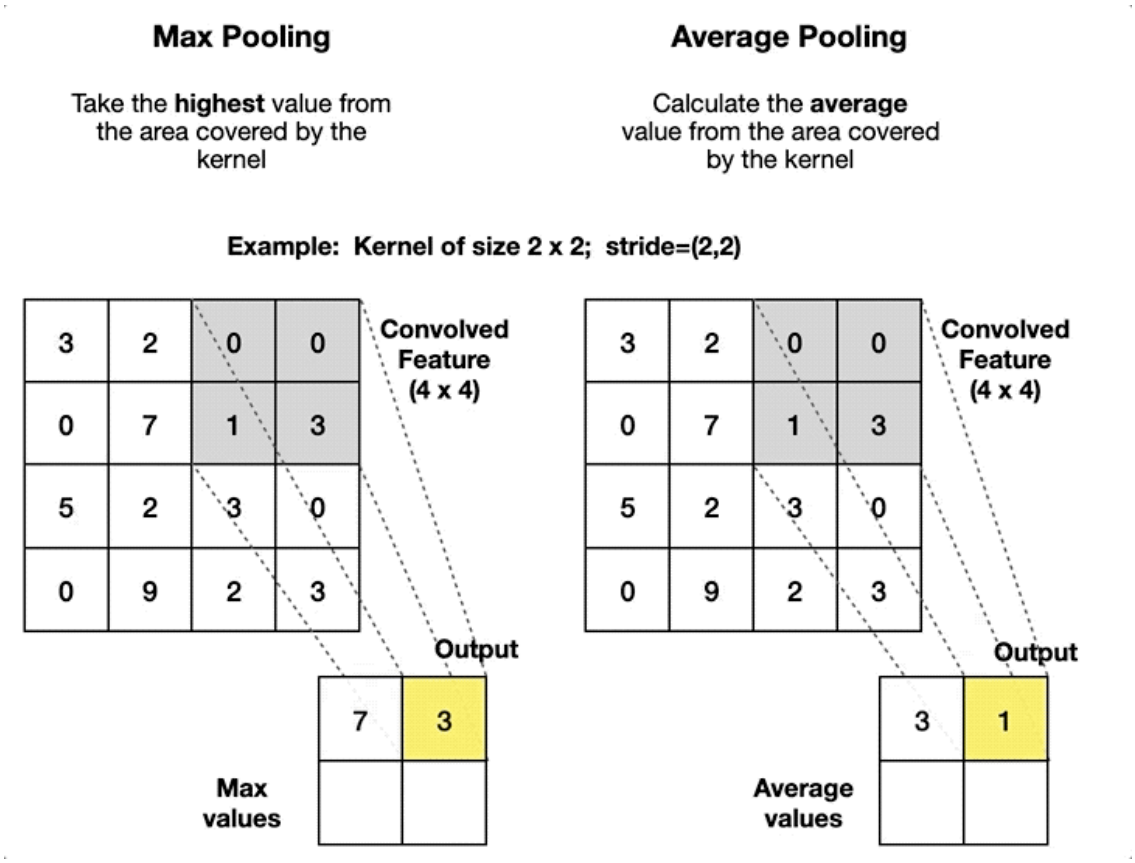
-1	-1	-1
-1	8	-1
-1	-1	-1

commonly used edge detection filter.

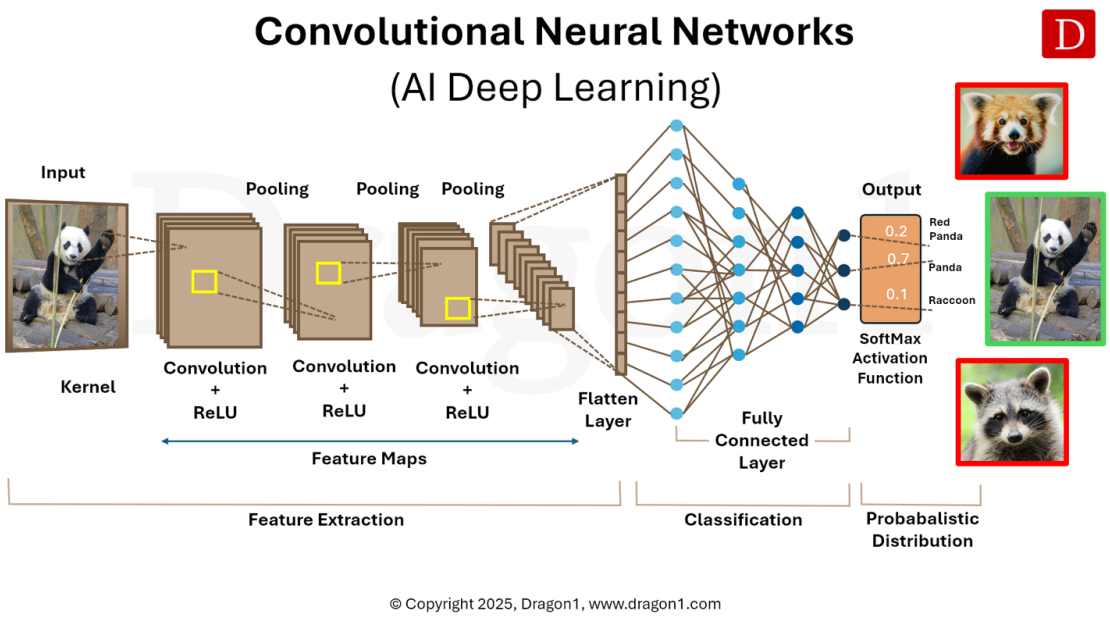
The idea here is that when the pixel is similar to all its neighbors, they should cancel each other, giving a value of 0. Therefore, the more similar the pixels, the darker the part of the image, and the more different they are the lighter it is.

Still, processing the image in a neural network is computationally expensive due to the number of pixels that serve as input to the neural network.

- **Pooling**: the size of the input is reduced by sampling from regions in the input. Pixels that are next to each other belong to the same area in the image, which means that they are likely to be similar.
  - **Max-pooling**: the selected pixel is the one with the highest value of all others in the same region.
  - **Average-pooling** computes the average value of its l inputs.
  - Max pooling selects the maximum value in a window, emphasizing sharp, prominent features like edges, making it ideal for object detection and faster training. Average pooling computes the mean value, producing smoother, more general features, which can reduce overfitting but may lose sharp details.



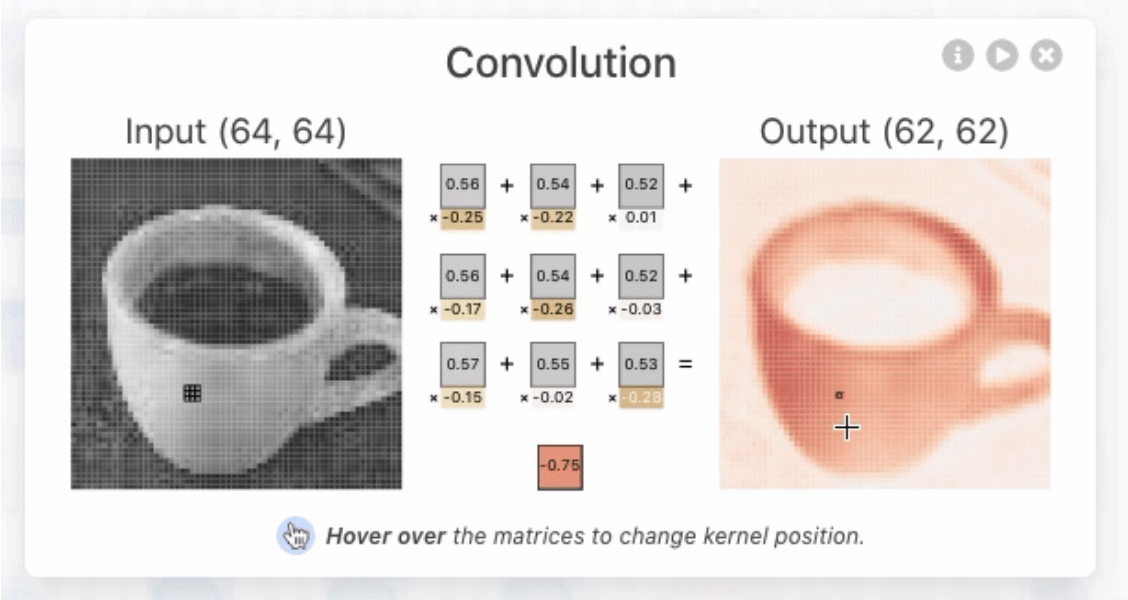
Convolutional Neural Networks $CNNs$

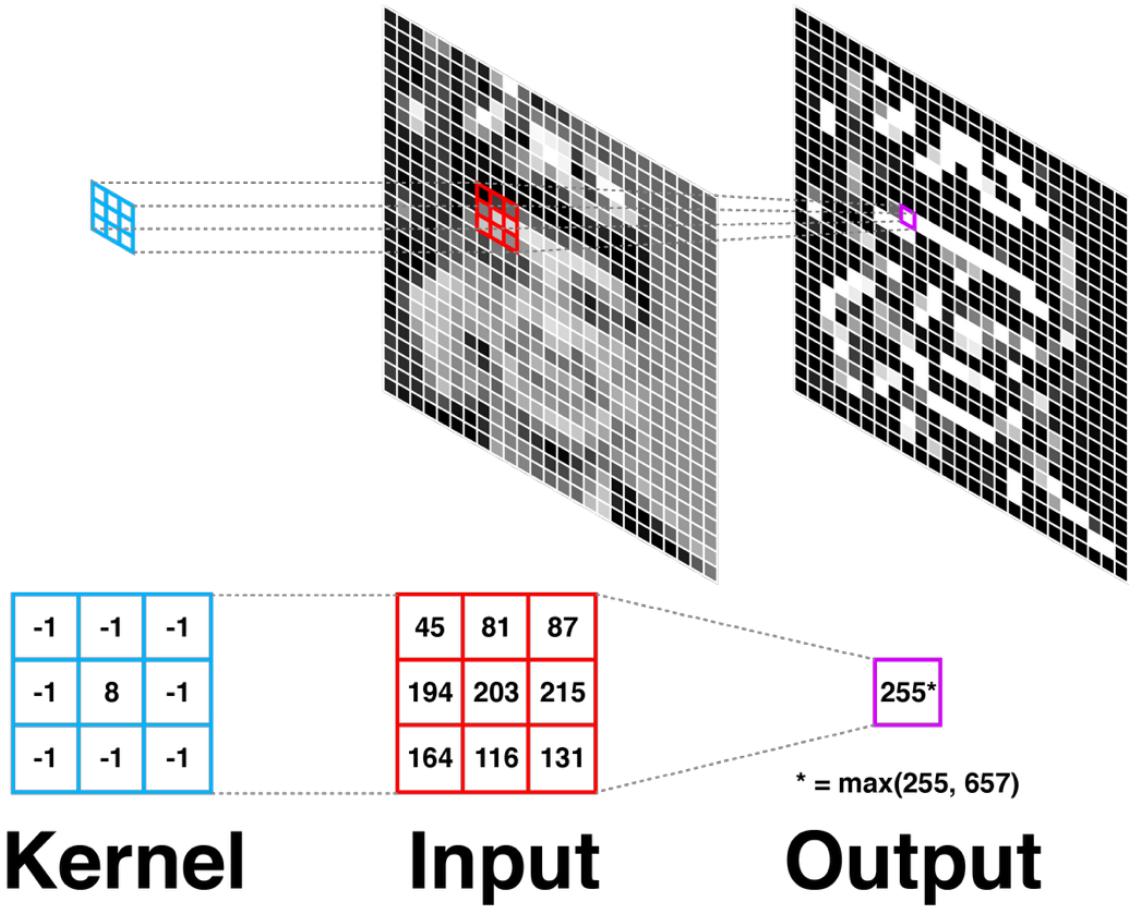


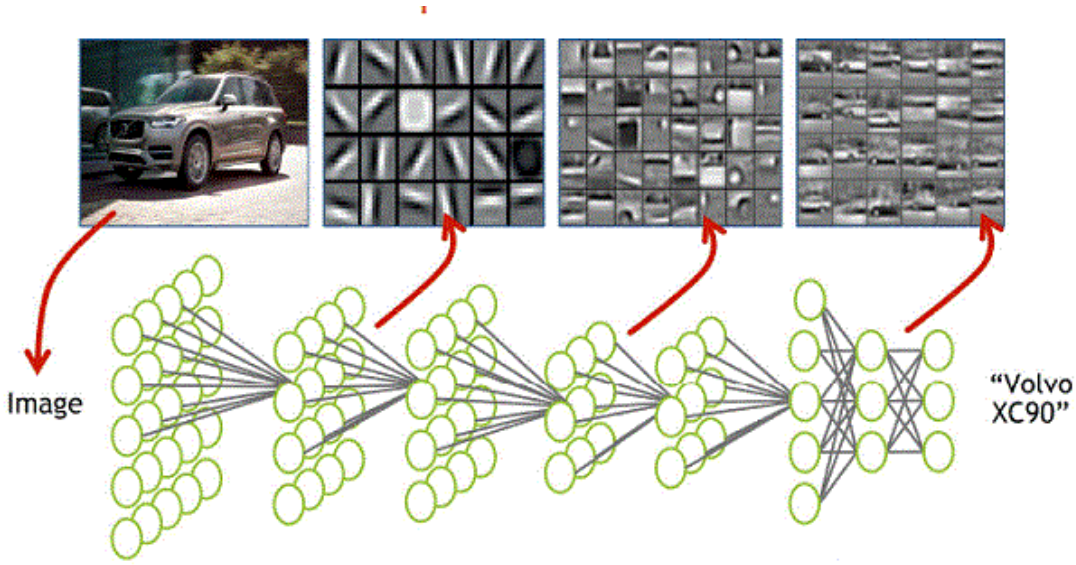
A Convolutional Neural Network architecture consists of multiple layers that work together to understand more about an image gradually. The combination of convolution, ReLU, and pooling is often repeated multiple times.

This allows the network first to learn simple features $lines$ and then progressively more complex shapes $eyesorobjects$. Common layers include:

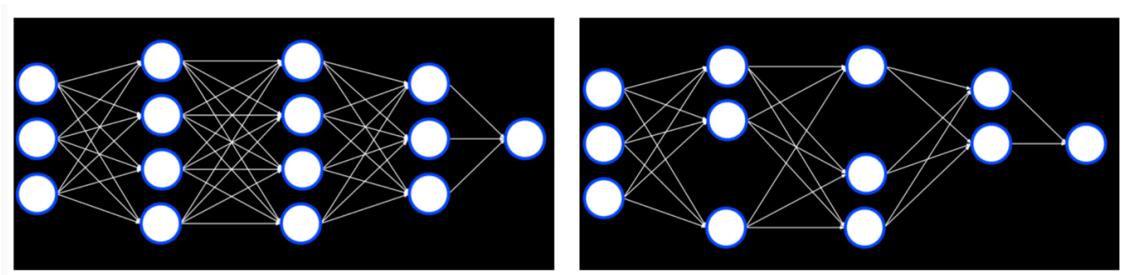
1. **Input Image**: Taking in images as pixels, looking at channels: grayscales and RGB, and dimensions.
2. **Convolutional Layer:** A small filter slides over an image and recognizes patterns, such as edges or corners. This produces a feature map: a map of where specific patterns were found.
3. **Activation Function** $ReLU - RectifiedLinearUnits$ - Activates the network by allowing only positive signals to pass: $f(x) = \max(0, x)$. Adds non-linearity, enabling the network to learn complex patterns.
4. **Pooling Layer**: Simplifies the data by reducing its size, e.g., using max pooling $pickingthelargestvalueinaregion$. For example, of $[4, 7, 1, 5]$, only the 7 is retained. Helps reduce data size, increases speed, and helps prevent overfitting. Repeated Conv + ReLU + Pooling: Often used multiple times to capture increasingly abstract features $fromedgestofullobjects$.
5. **Fully Connected / Flattening Layer**: Flattens 2D feature maps into a 1D vector for feeding into the final layers. Combines all learned information to make a prediction.
6. **Output Layer:** Produces the final result, often using a softmax function to calculate probabilities.


Convolution
Input (64, 64)
Output (62, 62)
0.56 × -0.25 + 0.54 × -0.22 + 0.52 × 0.01 +
0.56 × -0.17 + 0.54 × -0.26 + 0.52 × -0.03 +
0.57 × -0.15 + 0.55 × -0.02 + 0.53 × -0.28 =
-0.75
Hover over the matrices to change kernel position.


-1 -1 -1
-1 8 -1
-1 -1 -1
45 81 87
194 203 215
164 116 131
255*
* = max(255, 657)
Kernel
Input
Output



Generalization



Dropout

At each step of training, dropout applies one step of back-propagation learning to a new version of the network that is created by deactivating a randomly chosen subset of the units.

1. Introduce noise that provides robustness
2. Approximation of large ensemble of thinned networks
3. Paying attention to all features of the example rather than focusing on just a few

Usually makes it harder to fit the training set, it is usually necessary to use a larger model and to train it for more iterations.

Data augmentation

Provide multiple version of the same input $for example for images: add noise, crop, rotate, scale$ to increase the size of the training data and the robustness to the various transformations applied.

Tensors

Multi-dimensional arrays of any dimension - they keep track of the "shape" of the data as it progresses through layers of the network. Describing the CNN in terms of tensors and tensor operators, a deep learning package can generate compiled code that is highly optimized for the underlying computational substrate.

The support we are trying on 256×256 RGB images with a minibatch size of 64. The input will be a 4-dimensional tensor of size $256 \times 256 \times 3 \times 64$. Then we apply 96 kernels of size $5 \times 5 \times 3$ with a stride of 2 in both x and y dimensions. This gives an output tensor of size $128 \times 128 \times 96 \times 64$. Such a tensor is called a **feature map** - note no dedicated colour channels but colour information has been incorporated if the learning algorithm finds it useful for the final predictions of the network.

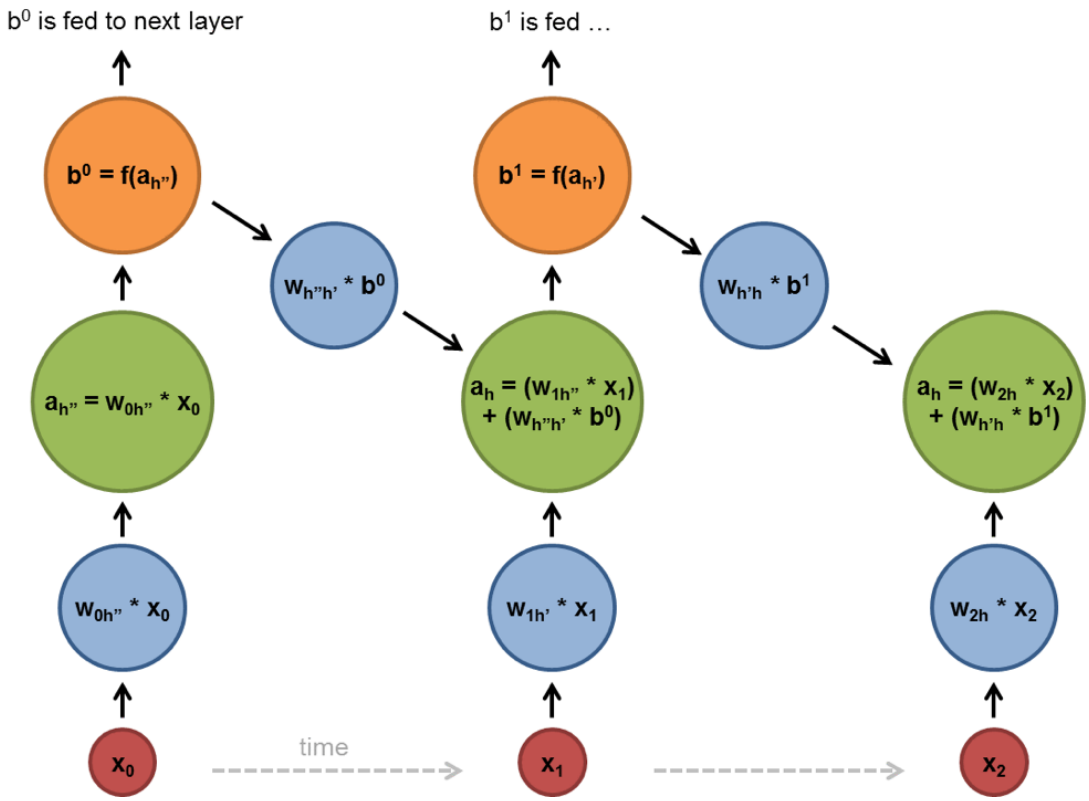
Graphical Processing units $GPUs$ are specialized hardware for graphics operations that can be used to perform tensor operations. Tensor processing units $TPUs$ are specialized hardware for computing tensor operations that optimize for speed and throughput rather than high numerical precision.

4. Recurrent Neural Networks

Feed-Forward Neural Networks are the type of neural networks that we have discussed so far, where input data is provided to the network, which eventually produces some output.

As opposed to that, Recurrent Neural Networks consist of a non-linear structure, where the network uses its own output as input. For example, some RNNs are capable of describing the content of an image with words in a sentence. This is different from classification in that the output can be of varying length based on the properties of the image. While feed-forward neural networks are incapable of varying the number of outputs, recurrent neural networks are capable to do that due to their structure. In the captioning task, a network would process the input to produce an output, and then continue processing from that point on, producing another output, and repeating as much as necessary.

Recurrent neural networks are helpful in cases where the network deals with sequences and not a single individual object. Above, the neural network needed to produce a sequence of words. However, the same principle can be applied to analyzing video files, which consist of a sequence of images, or in translation tasks, where a sequence of inputs $wordsinthesourcelanguage$ is processed to produce a sequence of outputs $wordsinthetargetlanguage$.



source: UBC wiki

Applications of Recurrent Neural Networks:

1. **Natural Language Processing NLP:** RNNs have revolutionized the field of NLP, enabling tasks like language translation, sentiment analysis, and text generation. Explore how the sequential nature of language makes RNNs particularly well-suited for these applications. Delve into case studies of language models like OpenAI's GPT $GenerativePre-trainedTransformer$ that leverage RNNs for advanced natural language understanding.

2. **Time Series Analysis:** Time series data, common in finance, weather forecasting, and stock market analysis, requires models capable of capturing temporal dependencies. Illustrate how RNNs excel in predicting future values based on historical data, demonstrating their efficacy in time series forecasting.

3. **Speech Recognition:** Investigate the role of RNNs in speech recognition systems. Explore how these networks can be trained to recognize and transcribe spoken words, contributing to advancements in virtual assistants, voice-controlled devices, and more.